

ElderlyCare

Secure emergency information, care workflows, and privacy-aware health support

MERN STACK	QR EMERGENCY ACCESS	ADMIN RBAC
------------	---------------------	------------

Complete project handbook

Architecture | Features | Workflows | APIs | Installation | Deployment | Security

Prepared from the repository implementation
github.com/kaif8077/elderly-care

Table of contents

Live application	5
Problem and solution	5
Main users	5
Elderly account owner	5
Emergency responder	6
Guardian, caregiver, or doctor	6
Administrator	6
Implemented features	6
Authentication and account recovery	6
Medical profile	6
Profile photograph	6
QR emergency access and ID card	7
Privacy controls	7
Emergency alerts	7
Health recommendations	7
Secure medical documents	8
Medical reports	8
Care workflow	8
Contact page	9
Administration	9
Accessibility and responsive UI	9
Application workflows	9
Registration workflow	9
Normal login workflow	10
Medical-profile workflow	10
QR workflow	10
Emergency-alert workflow	10
Admin workflow	11
Technology stack	11
Frontend	11
Backend	12
Architecture	12
Frontend architecture	12
Backend architecture	12
Project structure	13
Database models	14
API overview	14
Health and status	14
User authentication	15

Medical profile and photograph	15
QR and emergency access	15
Recommendations and reports	16
Documents and care workflow	16
Contact and compatibility alert	16
Admin authentication and management	17
Local installation	17
Prerequisites	17
1. Clone the repository	17
2. Install backend dependencies	18
3. Create the backend environment file	18
4. Start MongoDB	18
5. Configure Resend	18
6. Seed the development admin	18
7. Start the backend	18
8. Install frontend dependencies	19
9. Create the frontend environment file	19
10. Start the frontend	19
Environment variables	19
Required backend variables	19
Frontend variables	20
Available commands	20
Repository root	20
Backend	20
Frontend	20
Testing	20
Deployment	21
Backend on Render	21
Frontend on Vercel	21
Cross-origin admin sessions	22
Security and privacy	22
Implemented safeguards	22
Secret handling	22
Medical-data boundary	23
Known limitations	23
Troubleshooting	23
Frontend still calls localhost:5000 after deployment	23
Browser reports a CORS error	23
Admin login fails after deleting MongoDB data	23
Admin returns ADMIN_AUTH_REQUIRED	24
Admin returns ACCOUNT_NOT_ACTIVE or ADMIN_ACCESS_DENIED	24

Registration email is not delivered 24

Medical profile cannot be finalized 24

QR page says the profile is unavailable 24

Render health endpoint returns 503 24

License 24

Maintainer 24

ElderlyCare is a full-stack MERN healthcare-support application that helps older adults and their families organize essential medical information and make a limited emergency summary available through a secure, revocable QR-based ID card.

The application combines email-verified registration, structured medical profiles, privacy-controlled emergency access, geolocation-based alerts, protected document storage, saved health recommendations, PDF generation, caregiver workflows, and a role-protected administration portal.

Important: ElderlyCare provides emergency-information and general wellness support. It is not a diagnostic system, a substitute for professional medical advice, or a claim of legal or healthcare-regulatory compliance. A production healthcare deployment requires independent security, privacy, legal, and clinical review.

Live application

- Frontend: <https://kaif8077-elderly-care.vercel.app>
- Backend health check: <https://elderly-care-zuq9.onrender.com/api/health>
- Repository: <https://github.com/kaif8077/elderly-care>

The Render free tier may sleep when inactive, so the first backend request can take longer than later requests.

Problem and solution

During a fall, accident, or medical emergency, an older adult may be unable to communicate their blood group, allergies, medications, conditions, mobility needs, preferred language, or emergency contacts. That information is often distributed across prescriptions, reports, family members, and different healthcare providers.

ElderlyCare creates one controlled emergency-readiness workflow:

```
Register and verify email
      v
Create a structured medical profile
      v
Choose privacy and consent settings
      v
Generate an ElderlyCare ID card and secure QR token
      v
Responder scans the QR code
      v
Backend returns only permitted emergency fields
      v
Responder calls a contact or sends a location-based alert
      v
Emergency contact acknowledges or resolves the alert
```

The QR code does not contain the complete medical record. It contains a URL with an opaque, non-guessable token that can be revoked and regenerated.

Main users

Elderly account owner

The account owner can create a medical profile, upload a photograph and medical documents, manage emergency contacts, generate a QR card, view saved recommendations, configure privacy, and manage caregiver relationships.

Emergency responder

A responder can scan an active QR code without logging in, view only the permitted emergency summary, call listed contacts, provide an optional responder message, share browser-approved location, and create an emergency alert. A responder cannot edit the profile.

Guardian, caregiver, or doctor

The care workflow supports invitations and scoped permissions such as profile access, profile updates, alert receipt, and reminder management. Relationships can be pending, active, or revoked.

Administrator

An administrator uses a separate secure login and can view dashboard statistics, search users, inspect authorized user details, view ID cards, manage account status, read contact submissions, and review audit logs. Backend role and permission middleware protects every admin operation.

Implemented features

Authentication and account recovery

- Email OTP verification during registration
- Automatic login after successful registration
- Email-and-password login without login OTP
- bcrypt password hashing
- Seven-day signed user JWTs
- Session restoration through </api/auth/me>
- Session revocation through [sessionVersion](#)
- Password-reset OTP and short-lived reset token
- Active, inactive, suspended, and archived account states
- Soft deletion and restoration support

Medical profile

- Section-by-section draft saving
- Final completion validation
- Automatically generated 12-digit ElderlyCare ID
- First name, last name, date of birth, gender, and blood group
- Height, weight, diet preference, marital status, and mobility status
- Preferred languages and fall-risk information
- Address and phone number
- Multiple emergency contacts with relationship and priority data
- Conditions, allergies, medications, and current symptoms
- Doctor and preferred-hospital details
- Insurance summary
- Required profile photograph before final completion
- Profile review and consent fields

Profile photograph

- Authenticated multipart upload

- JPEG, PNG, and WebP support
- Maximum stored size of 3 MB
- File-signature validation rather than trusting only the browser MIME type
- MongoDB GridFS storage
- Authenticated download and removal
- Safe photo metadata in normal profile responses

QR emergency access and ID card

- Opaque random QR access token
- Revocable and regenerable QR records
- User-owned QR generation
- Mobile emergency profile route
- Backend-enforced public emergency projection
- QR access logging with automatic retention expiry
- ElderlyCare ID card with profile photograph and QR code
- Responsive card preview
- PDF card download
- Admin ID-card inspection and QR management APIs

Privacy controls

Profile fields can use one of these access levels:

- `public_emergency`
- `emergency_contacts`
- `owner_only`
- `hidden`

The backend applies these rules when building public emergency responses. Hiding a field in React is not treated as access control.

Emergency alerts

- Public alert creation through an active QR token
- Situation types including medical emergency, fall, accident, person found, and lost/confused person
- Optional responder name, phone number, and message
- Browser-permission-based latitude, longitude, accuracy, and Google Maps link
- Resend email delivery to verified/eligible emergency contacts
- Delivery status tracking
- Alert acknowledgement workflow
- Received, calling, travelling, emergency-services-contacted, and resolved actions
- Duplicate-alert detection
- Request rate limiting
- One-way HMAC hashing of abuse-protection identifiers
- One-year automatic alert-record expiry

Health recommendations

- Profile-aware recommendation generation

- Consent check before generation
- Safe deterministic fallback recommendations enabled by default
- Optional external Hugging Face inference when explicitly enabled
- 24-hour recommendation cache
- Saved recommendation history
- Helpful/not-helpful feedback
- Authenticated PDF download
- General wellness boundary; no diagnosis or medication changes

`ENABLE_EXTERNAL_AI` is disabled by default so medical profiles stay inside ElderlyCare. The installed OpenAI SDK is not part of the active recommendation path in the current implementation.

Secure medical documents

- Prescription, report, insurance card, doctor note, vaccination, discharge summary, and identification categories
- PDF, JPEG, PNG, and WebP uploads
- Maximum document size of 5 MB
- File-signature validation
- MongoDB GridFS storage
- Owner-scoped listing and download
- Owner-only or emergency-contact access metadata
- Archive instead of immediate permanent deletion
- Preview, download, and delete/archive controls

Medical reports

- Authenticated report creation and history APIs
- Immutable medical-profile snapshots
- Incrementing report versions
- Latest-report marker
- Visible and hidden section lists
- Insurance hidden by default
- Server-generated emergency-summary PDF
- Admin verification and archive APIs

Historical report snapshots do not change when the current profile is updated.

Care workflow

- Care-center overview
- Emergency-contact verification emails
- Privacy updates
- Notification preferences
- Medication, appointment, profile-review, insurance-expiry, and contact-verification reminders
- Guardian, caregiver, and doctor invitations
- Scoped care-team permissions
- Invitation acceptance and relationship revocation

Contact page

- Name, email, phone number, and message submission
- Request validation and rate limiting
- Duplicate-protection metadata
- Resend notification delivery
- Admin contact-message listing

Administration

- Admin account seed script
- Separate admin login endpoint
- HTTP-only admin session cookie
- Eight-hour default admin session, configurable up to 24 hours
- Admin and super-admin role boundary
- Permission-based route protection
- Five-attempt login lockout with a 15-minute lock period
- Dashboard statistics
- Searchable, filterable, sortable, paginated users
- Authorized user-detail view
- Account activation, deactivation, archive, and restoration
- QR revoke and regenerate actions
- ID-card view and profile-photo access
- Contact-message management
- Admin audit logs
- Password-change endpoint with stronger password requirements

Accessibility and responsive UI

- React and Ant Design component system
- Responsive desktop and mobile navigation
- Mobile admin drawer
- Accessible labels and icon descriptions
- Loading, empty, success, and error states
- Large emergency action controls
- Keyboard-compatible Ant Design components
- Reduced-motion support
- Primary theme colour #0066ff
- Secondary/hover colour #ff6b00

Application workflows

Registration workflow

1. The user enters a name, email address, and password.
2. The frontend calls `POST /api/auth/register`.
3. The backend creates a six-digit registration OTP with a ten-minute expiry.

4. Resend emails the OTP.
5. The frontend submits the code to `POST /api/auth/verify-otp`.
6. The backend verifies the OTP and creates a 15-minute registration token.
7. The frontend submits the name, password, and registration token to `POST /api/auth/complete-registration`.
8. The backend hashes the password with bcrypt and creates a verified user.
9. A user JWT is returned and the user is redirected to the dashboard.

Normal login workflow

1. The user submits email and password to `POST /api/auth/login`.
2. The backend normalizes the email and compares the password using bcrypt.
3. Verification status, account status, deletion state, and session version are checked.
4. A signed JWT containing the user ID and session version is returned.
5. Protected frontend requests send the JWT as a Bearer token.
6. On page refresh, the frontend calls `GET /api/auth/me` to restore the session.

Medical-profile workflow

1. The user completes the personal-information section and uploads a profile photograph.
2. Each form step is saved independently through `POST /api/medical`.
3. Atomic draft updates validate only the submitted section and preserve previous sections.
4. The address and emergency-contact section requires at least one complete emergency contact.
5. Medical conditions, allergies, medications, and symptoms are stored as structured arrays.
6. The review step submits `finalize: true`.
7. The backend checks every mandatory field and changes the profile status to `completed`.
8. The profile page loads the completed data, recommendations, documents, QR card, and care settings.

QR workflow

1. An authenticated user requests a QR code through `POST /api/qr`.
2. The backend creates a cryptographically random token and a QR image.
3. The QR points to `/emergency/:token` rather than embedding medical data.
4. A responder scans the card and the frontend requests `/api/qr/public/:token`.
5. The backend checks the QR status, user status, profile availability, consent, and field visibility.
6. Only approved emergency information is returned.
7. Revoking the QR immediately disables the old token; regeneration creates a new token.

Emergency-alert workflow

1. The responder selects the situation type and optionally enters contact details and a message.
2. Location is requested through the browser Geolocation API; it is shared only after permission.
3. The public request is rate-limited and validated.
4. The backend hashes the request IP for abuse protection and checks recent duplicates.
5. An `EmergencyAlert` record is created with delivery status entries.
6. Eligible contacts receive an email containing limited context, the map link, and an acknowledgement link.
7. A contact can acknowledge the alert or mark it resolved.
8. QR access and alert events are retained for a limited period.

Admin workflow

1. A development admin is created with `npm run seed:admin` in the backend directory.
2. The admin submits credentials to `POST /api/admin/auth/login`.
3. The backend validates the password, role, account status, lockout status, and trusted browser origin.
4. A dedicated admin JWT is stored in an HTTP-only cookie.
5. Protected requests pass through admin authentication, role, and permission middleware.
6. Sensitive actions write an audit-log entry.
7. Logout clears the admin cookie.

Technology stack

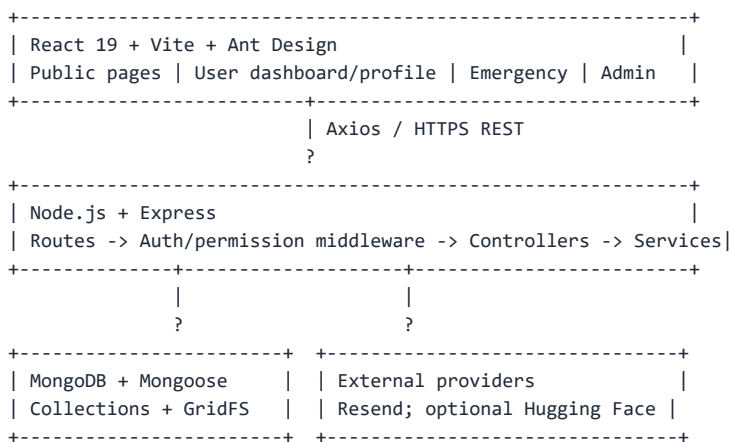
Frontend

Technology	Declared version	Use
React	19.x	Component-based UI
React DOM	19.x	Browser rendering
Vite	7.3.6	Development server and production bundling
React Router DOM	6.30.4	Public, protected, emergency, and admin routing
Ant Design	5.29.3	Forms, cards, tables, modals, layouts, feedback, and accessibility
Ant Design Icons	5.6.1	Consistent UI icons
Axios	1.8.x	REST API requests
React Toastify	11.x	User feedback notifications
jsPDF	4.2.1	Client-side ID-card PDF generation
html2canvas	1.4.1	Capturing the rendered ID card for PDF output
React Icons	5.5.x	Additional interface icons
Vitest	3.2.7	Frontend tests
Testing Library	16.2.x	React behaviour tests

Backend

Technology	Declared version	Use
Node.js	20.19+	JavaScript runtime
Express	4.21.2	REST API and middleware
MongoDB	6+ recommended	Document database and GridFS storage
Mongoose	8.12.x	Schemas, validation, indexes, and queries
bcrypt	6.x	Password hashing
jsonwebtoken	9.0.2	User and admin JWTs
Resend	6.17.1	OTP, alert, verification, and contact emails
Multer	2.2.x	Multipart file uploads
QRCode	1.5.4	QR image generation
PDFKit	0.17.2	Server-side recommendation and report PDFs
CORS	2.8.5	Trusted frontend-origin control
dotenv	16.4.7	Environment configuration
Nodemon	3.1.9	Backend development reloads

Architecture



Frontend architecture

- [pages/](#) contains public and authenticated user screens.
- [components/](#) contains shared navigation, forms, QR, recommendations, documents, footer, and ID-card UI.
- [context/](#) stores normal-user authentication state.
- [admin/](#) contains isolated admin layout, pages, API client, session context, and route guards.
- [theme/](#) contains global Ant Design theme overrides and responsive shared styles.
- [services/](#) contains reusable frontend API helpers.

Backend architecture

- [routes/](#) maps HTTP endpoints to middleware and controllers.
- [middleware/](#) implements authentication, authorization, rate limiting, trusted origins, uploads, and security headers.

- [controllers/](#) validate request context and shape responses.
- [services/](#) contain reusable business logic, email delivery, storage, reports, sessions, and admin queries.
- [models/](#) define MongoDB schemas, enums, indexes, and retention rules.
- [scripts/](#) contains the idempotent admin seed process.
- [tests/](#) contains backend security, route, data-model, PDF, privacy, and workflow tests.

Project structure

```
elderly-care/
+-- backend/
|   +-- config/           # MongoDB connection
|   +-- controllers/      # User, medical, QR, alert, admin controllers
|   +-- middleware/       # Auth, RBAC, rate limits, uploads, headers
|   +-- models/           # Mongoose models
|   +-- routes/           # REST API modules
|   +-- scripts/seedAdmin.js # Secure admin seed
|   +-- services/         # Business and provider services
|   +-- tests/            # Node test suite
|   +-- .env.example
|   +-- app.js            # Express application
|   +-- server.js         # HTTP server entry
|   +-- package.json
+-- frontend/
|   +-- public/           # Static public files
|   +-- src/
|   |   +-- admin/        # Admin layout, pages, auth, API client
|   |   +-- assests/      # Existing image assets
|   |   +-- components/   # Shared user-facing components
|   |   +-- context/      # User authentication context
|   |   +-- pages/        # Public, user, and emergency pages
|   |   +-- services/     # Frontend API services
|   |   +-- theme/        # Shared Ant Design styles
|   |   +-- App.js        # Route definitions
|   |   +-- index.js      # React entry
|   +-- vercel.json       # SPA route rewrite
|   +-- vite.config.js
|   +-- package.json
+-- package.json          # Root verification shortcuts
+-- README.md
```

The existing directory is named [assests](#); keep that spelling when referencing current imports unless the directory and every import are migrated together.

Database models

Model	Responsibility
User	Credentials, verification, roles, permissions, status, lockout, and session revocation
MedicalProfile	Structured personal, emergency, medical, insurance, visibility, and consent data
QRCode	Opaque token, generated QR data, status, owner, and revocation metadata
QrAccessLog	Privacy-limited QR events with 180-day automatic expiry
EmergencyAlert	Situation, optional responder context, location, delivery, acknowledgement, and one-year expiry
MedicalDocument	Owner-scoped document metadata and protected GridFS identifier
MedicalReport	Versioned immutable emergency-summary snapshots
HealthRecommendation	Saved recommendation output, source summary, generator, and feedback
RecommendationCache	24-hour cached health and first-aid output
Otp	Hashed, purpose-bound, expiring verification codes
NotificationPreference	Email, push, Telegram, call, and SMS-fallback preferences
Reminder	Medication, appointment, review, expiry, and verification reminders
CareRelationship	Guardian, caregiver, and doctor invitations and permissions
Contact	Public contact submissions and delivery state
AuditLog	Sensitive admin and lifecycle action history
LocationAlert	Legacy location-alert records
ExerciseDietRecommendation	Legacy exercise/diet recommendation records

API overview

All request and response bodies use JSON unless an upload endpoint specifies multipart form data.

Health and status

Method	Endpoint	Authentication	Purpose
GET	/api/health	Public	Backend and database readiness
GET	/api	Public	Basic API information

User authentication

Method	Endpoint	Authentication	Purpose
POST	/api/auth/register	Public	Send registration OTP
POST	/api/auth/verify-otp	Public	Verify registration OTP
POST	/api/auth/complete-registration	Registration token	Create verified account
POST	/api/auth/login	Public	Email/password login
GET	/api/auth/me	Bearer token	Restore current user
POST	/api/auth/forgot-password	Public	Send password-reset OTP
POST	/api/auth/verify-reset-otp	Public	Verify reset OTP
POST	/api/auth/reset-password	Reset token	Save new password

Medical profile and photograph

Method	Endpoint	Authentication	Purpose
POST	/api/medical	User Bearer token	Create, save, or finalize profile
GET	/api/medical/:userId	Owner/admin Bearer token	Read profile
POST	/api/medical/:userId/photo	Owner Bearer token	Upload profile photograph
GET	/api/medical/:userId/photo	Owner/admin Bearer token	Download photograph
DELETE	/api/medical/:userId/photo	Owner Bearer token	Remove photograph

QR and emergency access

Method	Endpoint	Authentication	Purpose
POST	/api/qr	User Bearer token	Generate secure QR
GET	/api/qr/:userId	Owner Bearer token	Get current QR
GET	/api/qr/access/:token	Public token	Redirect to emergency page
GET	/api/qr/public/:token	Public token	Get filtered emergency summary
POST	/api/emergency-alerts/public/:token	Public token, rate-limited	Create emergency alert
PATCH	/api/emergency-alerts/acknowledge/:token	Public acknowledgement token	Update alert acknowledgement

Legacy scanner-OTP routes remain registered for compatibility, but the current basic emergency page uses the secure token and privacy-filtered public response.

Recommendations and reports

Method	Endpoint	Authentication	Purpose
POST	/api/recommendations/health	User Bearer token	Generate and save recommendation
GET	/api/recommendations/health	User Bearer token	List recommendation history
GET	/api/recommendations/health/:id/download	User Bearer token	Download recommendation PDF
PATCH	/api/recommendations/health/:id/feedback	User Bearer token	Save recommendation feedback
GET	/api/recommendations/firstaid/:profileId	Owner Bearer token	Generate first-aid guidance
POST	/api/medical-reports	User Bearer token	Generate versioned report
GET	/api/medical-reports	User Bearer token	List reports
GET	/api/medical-reports/latest	User Bearer token	Get latest report
GET	/api/medical-reports/:reportId	Owner Bearer token	Get report snapshot
GET	/api/medical-reports/:reportId/download	Owner Bearer token	Download report PDF

Documents and care workflow

Method	Endpoint	Authentication	Purpose
GET	/api/medical-documents	User Bearer token	List owned documents
POST	/api/medical-documents	User Bearer token	Upload protected document
GET	/api/medical-documents/:id/download	Owner Bearer token	Download document
DELETE	/api/medical-documents/:id	Owner Bearer token	Archive document
GET	/api/care/overview	User Bearer token	Load care-center data
PATCH	/api/care/privacy	User Bearer token	Update privacy settings
PATCH	/api/care/notification-preferences	User Bearer token	Update channels
POST	/api/care/reminders	User Bearer token	Create reminder
PATCH	/api/care/reminders/:id	User Bearer token	Update reminder
POST	/api/care/care-team/invitations	User Bearer token	Invite care member
POST	/api/care/care-team/invitations/:token/accept	User Bearer token	Accept invitation
DELETE	/api/care/care-team/:id	User Bearer token	Revoke relationship

Contact and compatibility alert

Method	Endpoint	Authentication	Purpose
POST	/api/contact	Public, rate-limited	Submit contact message
POST	/api/send-sms	Public compatibility route	Send the legacy alert payload by email

Despite its legacy route name, [/api/send-sms](#) currently sends a Resend email and does not silently send SMS.

Admin authentication and management

Admin endpoints use an HTTP-only cookie and backend permission checks.

Method	Endpoint	Permission/purpose
POST	/api/admin/auth/login	Trusted-origin admin login
GET	/api/admin/auth/me	Restore admin session
POST	/api/admin/auth/logout	End admin session
POST	/api/admin/auth/change-password	Change admin password and revoke old session
GET	/api/admin/dashboard	dashboard.read
GET	/api/admin/users	users.read
GET	/api/admin/users/:userId	profiles.read
PATCH	/api/admin/users/:userId/status	users.update
DELETE	/api/admin/users/:userId	users.archive
POST	/api/admin/users/:userId/restore	users.restore
GET	/api/admin/id-cards/:userId	idCards.read
GET	/api/admin/id-cards/:userId/photo	idCards.read
POST	/api/admin/id-cards/:userId/regenerate	qr.regenerate
POST	/api/admin/id-cards/:userId/revoke	qr.revoke
GET	/api/admin/audit-logs	auditLogs.read
GET	/api/admin/contacts	Authorized admin contact listing

Admin report APIs also exist for authenticated report listing, download, verification, regeneration, and archiving, although they are not currently exposed as a main admin navigation item.

Local installation

Prerequisites

Install or prepare:

- Node.js [20.19.0](#) or newer
- npm
- MongoDB 6+ locally, or a MongoDB Atlas connection string
- A Resend account for OTP and notification emails
- Git

1. Clone the repository

```
git clone https://github.com/kaif8077/elderly-care.git
cd elderly-care
```

2. Install backend dependencies

```
cd backend
npm install
```

3. Create the backend environment file

PowerShell:

```
Copy-Item .env.example .env
```

macOS/Linux:

```
cp .env.example .env
```

Edit [backend/.env](#) with your own values. For Vite local development, use:

```
FRONTEND_URL=http://localhost:5173
RENDER_BACKEND_URL=http://localhost:5000
```

Do not use Markdown link syntax, quotes copied from documentation, or trailing slashes in origin values.

4. Start MongoDB

For local MongoDB, make sure the database service is running and use a connection such as:

```
MONGO_URI=mongodb://127.0.0.1:27017/elderly-care
```

For MongoDB Atlas, create a database user, allow your current IP address, and paste the Atlas connection string into [MONGO_URI](#).

5. Configure Resend

1. Create a Resend account.
2. Create an API key.
3. For testing, use Resend's permitted onboarding sender and recipient rules.
4. For production, verify your own domain and sender address.
5. Set [RESEND_API_KEY](#), [RESEND_FROM_EMAIL](#), and [SYSTEM_NOTIFICATION_EMAIL](#).

6. Seed the development admin

From [backend/](#):

```
npm run seed:admin
```

The seed process is idempotent: it checks for an existing admin and does not create duplicates. It hashes the password before storage and refuses to silently promote an existing normal account using the same email.

The example password is for local testing only. Use a strong unique password before deployment.

7. Start the backend

Development with automatic restart:

```
npm run server
```

Production-style start:

```
npm start
```

The backend runs at <http://localhost:5000> by default.

Verify it:

```
http://localhost:5000/api/health
```

8. Install frontend dependencies

Open a second terminal:

```
cd frontend
npm install
```

9. Create the frontend environment file

Create `frontend/.env`:

```
VITE_BACKEND_URI=http://localhost:5000
```

Only variables prefixed with `VITE_` are available to frontend code. Never place secrets, database credentials, Resend keys, JWT secrets, or private medical information in a `VITE_` variable.

10. Start the frontend

```
npm run dev
```

Open:

```
http://localhost:5173
```

Admin login is available at:

```
http://localhost:5173/admin/login
```

Environment variables

Required backend variables

```
# Database
MONGO_URI=mongodb://127.0.0.1:27017/elderly-care

# Use different long random values
JWT_SECRET=replace_with_a_long_random_user_session_secret
ADMIN_JWT_SECRET=replace_with_a_different_long_random_admin_secret
ALERT_HASH_SECRET=replace_with_a_third_random_alert_hash_secret

# Admin session
ADMIN_SESSION_HOURS=8
ADMIN_COOKIE_SAME_SITE=lax

# Used only by npm run seed:admin
ADMIN_EMAIL=elderlycare@gmail.com
ADMIN_PASSWORD=replace_with_a_strong_development_password
ADMIN_FORCE_PASSWORD_CHANGE=true

# Exact allowed frontend origin; comma-separate multiple origins if required
FRONTEND_URL=http://localhost:5173
RENDER_BACKEND_URL=http://localhost:5000

# Email
RESEND_API_KEY=re_your_key
RESEND_FROM_EMAIL=onboarding@resend.dev
SYSTEM_NOTIFICATION_EMAIL=your_notification_recipient@example.com

# Recommendation provider
ENABLE_EXTERNAL_AI=false
# Required only when ENABLE_EXTERNAL_AI=true
HF_API_KEY=your_hugging_face_token

# Runtime
NODE_ENV=development
PORT=5000
```

Generate secrets locally instead of reusing passwords. For example:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

Run that command separately for `JWT_SECRET`, `ADMIN_JWT_SECRET`, and `ALERT_HASH_SECRET`.

Frontend variables

`VITE_BACKEND_URI=http://localhost:5000`

Vite reads environment variables when the frontend starts or builds. Restart the Vite server after changing `.env`.

Available commands

Repository root

Command	Purpose
<code>npm test</code>	Run the backend Node test suite
<code>npm run build</code>	Create the frontend production build
<code>npm run audit:production</code>	Audit production backend dependencies

Backend

Command	Purpose
<code>npm start</code>	Start <code>server.js</code> with Node
<code>npm run server</code>	Start with Nodemon
<code>npm test</code>	Run all files under <code>backend/tests</code>
<code>npm run seed:admin</code>	Create or verify the configured admin account

Frontend

Command	Purpose
<code>npm run dev</code>	Start the Vite development server
<code>npm start</code>	Alias for the Vite development server
<code>npm run build</code>	Build optimized assets into <code>frontend/dist</code>
<code>npm run preview</code>	Preview the production build locally
<code>npm test</code>	Run Vitest once

Testing

Run the complete available verification from the repository root:

```
npm test
npm --prefix frontend test
npm run build
npm run audit:production
```

The backend tests cover areas including:

- User and admin session claims

- HTTP-only admin-cookie behaviour
- RBAC and permission boundaries
- Trusted-origin protection
- Account status and session revocation
- Admin query pagination and filtering
- Medical-profile validation and draft updates
- Public QR privacy projection
- Opaque and revocable QR records
- Emergency-alert hashing and retention
- Document content-signature validation
- Report snapshots, versions, and PDFs
- Recommendation ownership, history, and PDF output
- Contact submission validation
- Care-workflow route protection

The frontend tests cover route rendering and the main public, user, emergency, and admin application boundaries.

Deployment

Backend on Render

Create a Node web service using the repository's [backend](#) directory.

Recommended settings:

```
Root directory: backend
Build command: npm install
Start command: npm start
Health check path: /api/health
```

Add every backend environment variable in Render. Production values should include:

```
NODE_ENV=production
FRONTEND_URL=https://your-vercel-domain.vercel.app
RENDER_BACKEND_URL=https://your-render-service.onrender.com
```

Use exact HTTPS origins without a trailing slash. If multiple frontend domains are genuinely required, [FRONTEND_URL](#) supports a comma-separated allowlist.

After creating or clearing the production database, open the Render Shell and run:

```
npm run seed:admin
```

The admin account is stored in MongoDB; deleting the database also deletes the admin.

Frontend on Vercel

Create a Vercel project using the repository's [frontend](#) directory.

Recommended settings:

```
Framework preset: Vite
Root directory: frontend
Build command: npm run build
Output directory: dist
```

Add this Vercel environment variable:

```
VITE_BACKEND_URI=https://your-render-service.onrender.com
```

`frontend/vercel.json` rewrites client-side routes to `index.html`, allowing direct navigation to React Router pages.

After changing `VITE_BACKEND_URI`, trigger a new Vercel deployment because Vite embeds the value at build time.

Cross-origin admin sessions

The production frontend and backend normally use different origins. The backend therefore uses HTTPS-only, HTTP-only, cross-site admin cookies and validates the browser origin for state-changing admin requests.

For admin login to work:

- `FRONTEND_URL` must exactly match the deployed frontend origin.
- The backend and frontend must both use HTTPS.
- Axios admin requests must include credentials.
- The browser must accept the secure cross-site cookie.
- `ADMIN_JWT_SECRET` must remain stable between deployments.

Security and privacy

Implemented safeguards

- bcrypt password hashes instead of plain passwords
- Purpose-bound, expiring OTP records
- JWT expiration and session-version revocation
- Separate user and admin JWT secrets
- HTTP-only admin cookie
- Admin logout after repeated failures
- Backend role and permission enforcement
- Trusted-origin validation for sensitive admin mutations
- Restricted CORS allowlist
- Security headers and disabled Express identification header
- Owner-scoped medical, document, recommendation, and report queries
- Random revocable QR tokens instead of predictable profile IDs
- Explicit public emergency-field projection
- QR and emergency access retention limits
- Rate limits on public alerts, contact forms, acknowledgements, and admin login
- One-way alert abuse hashes
- File-size, MIME, and content-signature validation
- Protected GridFS files rather than public document URLs
- Soft deletion and account restoration
- Admin audit logging
- Immutable historical report snapshots

Secret handling

Never commit:

- `.env` files
- MongoDB connection strings

- JWT secrets
- Resend API keys
- Admin passwords
- Provider tokens
- Real patient information

If a key is accidentally shared in a chat, commit, screenshot, or issue, revoke it at the provider and create a new one. Removing it from a later commit is not enough because it may remain in Git history.

Medical-data boundary

The public QR view must remain a minimal emergency summary. Insurance policy information, protected documents, complete addresses, private phone numbers, audit records, and full medical history must not be added to public responses without an explicit privacy review and backend access rule.

Known limitations

- Email is a useful low-cost notification channel but is not guaranteed real-time emergency delivery.
- The compatibility endpoint named `/api/send-sms` currently sends email, not SMS.
- Push and Telegram channels exist in the data model but require provider integration before delivery.
- Optional external recommendation inference currently targets Hugging Face models; the installed OpenAI package is not used by the active controller.
- Recommendations are not clinically verified and must not diagnose or prescribe.
- A Render free instance can cold-start after inactivity.
- Normal user bearer-token storage has a larger XSS exposure than an HTTP-only cookie architecture.
- In-memory rate limiting is per backend process; a multi-instance production deployment should use a shared store such as Redis.
- MongoDB GridFS is suitable for this project, but large-scale production storage may be better served by private object storage with signed URLs.
- The frontend production bundle should be further optimized with route-level code splitting.
- Full browser end-to-end tests and production monitoring remain future improvements.
- The project does not claim HIPAA, GDPR, DPDP, or other regulatory compliance without professional review.

Troubleshooting

Frontend still calls `localhost:5000` after deployment

Set `VITE_BACKEND_URI` in Vercel and redeploy. Vite environment variables are embedded during the build.

Browser reports a CORS error

Check that `FRONTEND_URL` exactly matches the browser origin, including scheme and port but excluding paths and trailing slashes.

Local Vite example:

```
FRONTEND_URL=http://localhost:5173
```

Admin login fails after deleting MongoDB data

The admin is a database record. Recreate it against the same database used by the backend:

```
cd backend
npm run seed:admin
```

For the live database, run the command from the Render Shell or from a trusted environment configured with the production [MONGO_URI](#).

Admin returns [ADMIN_AUTH_REQUIRED](#)

Confirm that:

- Admin login completed successfully.
- The frontend request includes credentials.
- [FRONTEND_URL](#) and [RENDER_BACKEND_URL](#) are correct.
- Both deployed services use HTTPS.
- The admin cookie is present and not expired.

Admin returns [ACCOUNT_NOT_ACTIVE](#) or [ADMIN_ACCESS_DENIED](#)

Check that the account has the [admin](#) or [super_admin](#) role, [accountStatus](#) is [active](#), [isDeleted](#) is false, and the admin was seeded in the same database used by the running backend.

Registration email is not delivered

Check:

- [RESEND_API_KEY](#)
- [RESEND_FROM_EMAIL](#)
- Resend sender-domain verification
- Resend testing-recipient restrictions
- Backend logs for a safe provider error

Do not print or expose the API key while troubleshooting.

Medical profile cannot be finalized

Final completion requires all mandatory personal/contact fields, at least one complete emergency contact, and an uploaded profile photograph. Save each section first, upload the photo through the multipart endpoint, then submit the final review.

QR page says the profile is unavailable

The QR may be revoked, the account may be inactive, the profile may not exist, or the old QR token may have been replaced. Generate a new QR from the authenticated profile/dashboard.

Render health endpoint returns [503](#)

The backend process is running but MongoDB is not connected. Check [MONGO_URI](#), Atlas network access, database-user credentials, and Render logs.

License

The backend package currently declares the ISC license. Add a repository-level [LICENSE](#) file before treating the entire project as formally licensed for redistribution.

Maintainer

Developed and maintained by [Mohammad Kaif](#).